

AHDS - PYTHON01

Python Programming & Engineering Reference

PART 1 - CORE LANGUAGE

Chapters 1-20 • English / Français

Engineering focus / Orientation ingénierie
Precise language semantics first, then practical consequences for reliable software, automation and embedded/hardware-facing code.
D'abord une sémantique précise du langage, puis les conséquences pratiques pour les logiciels fiables, l'automatisation et le code en interaction avec le matériel.

Editorial note / Note éditoriale
Version-sensitive Python 3.15 material must be checked against the final official release documentation before publication.
Les éléments sensibles à la version Python 3.15 doivent être vérifiés dans la documentation officielle de la version finale avant publication.



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



AHDS | PYTHON-01 | PART 1 - CORE LANGUAGE | EN / FR

Table des matières / Contents

01. Python Overview & Engineering Applications
02. Program Structure & Execution Model
03. Lexical Structure & Syntax Fundamentals
04. Literals & Constants
05. Python 3.15 Language Additions - Overview & Cross-References
06. Variables, Names & Binding
07. Objects, Identity, Type & Value
08. Arithmetic Operators
09. Comparison Operators
10. Boolean & Logical Operations
11. Bitwise Operators & Engineering Bit Manipulation
12. Assignment & Assignment Expressions
13. Membership & Identity Operations
14. Conditional Expressions
15. Operator Precedence, Associativity & Evaluation Order
16. Simple Statements
17. Compound Statements, Blocks & Suites
18. Conditional Statements
19. Loops & Iterative Control Flow
20. Structural Pattern Matching



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 1 - Python Overview & Engineering Applications

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q1.1 What is Python and why is it useful in engineering? Python is a high-level, general-purpose language with dynamic typing, automatic memory management, a large standard library and a broad package ecosystem. Engineers use it for test automation, data acquisition, instrument control, analysis, report generation, build tools and host-side communication with embedded systems.	**Q1.1 Qu'est-ce que Python et pourquoi est-il utile en ingénierie ?** Python est un langage généraliste de haut niveau avec typage dynamique, gestion automatique de la mémoire, vaste bibliothèque standard et riche écosystème. Les ingénieurs l'utilisent pour l'automatisation des essais, l'acquisition de données, le contrôle d'instruments, l'analyse, la génération de rapports, les outils de construction et la communication avec les systèmes embarqués.
Q1.2 Is Python interpreted or compiled? Python source is normally parsed and compiled to code objects/bytecode, then executed by an implementation such as CPython. Calling Python only “interpreted” is therefore an oversimplification.	**Q1.2 Python est-il interprété ou compilé ?** Le code source Python est normalement analysé et compilé en objets de code/bytecode, puis exécuté par une implémentation comme CPython. Dire que Python est seulement « interprété » est donc une simplification.
Q1.3 Where does Python fit in an embedded project? Python commonly runs on the engineering PC as a test, configuration, automation or analysis layer, while C/C++ or assembly runs on the MCU. MicroPython can also run on supported microcontrollers.	**Q1.3 Quelle est la place de Python dans un projet embarqué ?** Python fonctionne souvent sur le PC d'ingénierie comme couche de test, de configuration, d'automatisation ou d'analyse, tandis que C/C++ ou l'assembleur s'exécute sur le MCU. MicroPython peut aussi fonctionner sur certains microcontrôleurs.

Engineering Notes / Notes d'ingénierie

EN	FR
• Distinguish language guarantees from CPython implementation details. • Do not assume desktop Python timing is deterministic or hard real-time. • Make units, ranges, signedness and fixed-width assumptions explicit when Python interacts with hardware.	• Distinguer les garanties du langage des détails propres à l'implémentation CPython. • Ne pas supposer que le temps d'exécution de Python sur PC est déterministe ou temps réel strict. • Rendre explicites les unités, plages, signes et largeurs de mots lors des échanges avec le matériel.

Shared engineering example - code remains English only

```
raw = instrument.read()
value = parse_measurement(raw)
result = "PASS" if 4.75 <= value <= 5.25 else "FAIL"
logger.info("rail_5v=%.3f V result=%s", value, result)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 2 - Program Structure & Execution Model

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q2.1 What happens when Python executes a source file? The implementation decodes the source, tokenizes and parses it, creates executable code objects, then executes the module's top-level statements in order. Imports may initialize additional modules.	**Q2.1 Que se passe-t-il lorsque Python exécute un fichier source ?** L'implémentation décode le source, le transforme en jetons, l'analyse, crée des objets de code exécutables puis exécute les instructions de niveau module dans l'ordre. Les imports peuvent initialiser d'autres modules.
Q2.2 What is a module's top-level code? It is code executed while the module body is initialized: assignments, imports, definitions and other statements not nested inside a function body.	**Q2.2 Qu'est-ce que le code de niveau supérieur d'un module ?** C'est le code exécuté pendant l'initialisation du module : affectations, imports, définitions et autres instructions qui ne sont pas imbriquées dans une fonction.
Q2.3 Why use if __name__ == '__main__'? It separates reusable definitions from the program entry point, preventing command-line or test code from running automatically when the module is imported.	**Q2.3 Pourquoi utiliser if __name__ == '__main__' ?** Cette garde sépare les définitions réutilisables du point d'entrée et évite d'exécuter automatiquement le code principal lors d'un import.

Engineering Notes / Notes d'ingénierie

EN	FR
• Keep reusable functions separate from entry-point code. • Imports can execute module initialization; avoid unnecessary top-level side effects. • Exceptions unwind execution frames until handled or the program terminates.	• Séparer les fonctions réutilisables du code de point d'entrée. • Un import peut exécuter l'initialisation d'un module; éviter les effets de bord inutiles au niveau module. • Les exceptions remontent les cadres d'exécution jusqu'à être traitées ou jusqu'à l'arrêt du programme.

Shared engineering example - code remains English only

```
def main():
    print("Engineering tool started")

if __name__ == "__main__":
    main()
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 3 - Lexical Structure & Syntax Fundamentals

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q3.1 What is the difference between a physical line and a logical line? A physical line ends with a line terminator. A logical line is the unit seen by the parser and may span several physical lines through permitted line joining.	**Q3.1 Quelle est la différence entre une ligne physique et une ligne logique ?** Une ligne physique se termine par une fin de ligne. Une ligne logique est l'unité vue par l'analyseur et peut couvrir plusieurs lignes physiques grâce aux règles de continuation.
Q3.2 How does indentation affect Python syntax? Indentation is syntactic. INDENT and DEDENT boundaries define suites and blocks. Inconsistent indentation can raise IndentationError or TabError and can also create incorrect control flow.	**Q3.2 Quel est l'effet de l'indentation sur la syntaxe de Python ?** L'indentation fait partie de la syntaxe. Les limites INDENT et DEDENT définissent les blocs. Une indentation incohérente peut provoquer IndentationError ou TabError et modifier la logique.
Q3.3 What source encoding is normally used? Modern Python source uses UTF-8 by default. An encoding declaration is rarely needed unless another supported encoding is intentionally required.	**Q3.3 Quel encodage du code source est normalement utilisé ?** Le code source Python moderne utilise UTF-8 par défaut. Une déclaration d'encodage est rarement nécessaire sauf si un autre encodage pris en charge est volontairement utilisé.

Engineering Notes / Notes d'ingénierie

EN	FR
• Prefer spaces and consistent indentation. • Use parentheses for multi-line expressions instead of fragile backslash continuation. • Conservative ASCII identifiers can improve interoperability in engineering tools.	• Préférer les espaces et une indentation cohérente. • Pour les expressions multilignes, préférer les parenthèses à la continuation fragile par barre oblique inverse. • Des identificateurs ASCII simples peuvent améliorer l'interopérabilité des outils d'ingénierie.

Shared engineering example - code remains English only

```
samples = (
   read_voltage(),
   read_current(),
   read_temperature(),
)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 4 - Literals & Constants

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q4.1 How are integer literals written in different bases? Decimal uses ordinary digits; binary uses 0b, octal 0o and hexadecimal 0x. For example, 42, 0b101010, 0o52 and 0x2A represent the same integer value.	**Q4.1 Comment écrit-on les littéraux entiers dans différentes bases ?** Le décimal utilise les chiffres ordinaires; le binaire 0b, l'octal 0o et l'hexadécimal 0x. Ainsi 42, 0b101010, 0o52 et 0x2A représentent la même valeur.
Q4.2 Is -10 a negative integer literal? No. The literal is 10; the minus sign is the unary negation operator applied to it.	**Q4.2 -10 est-il un littéral entier négatif ?** Non. Le littéral est 10; le signe moins est l'opérateur unaire de négation qui lui est appliqué.
Q4.3 What are important string/bytes literal forms? str literals represent Unicode text, bytes literals represent byte sequences, raw strings reduce backslash escaping, and f-strings embed evaluated replacement fields.	**Q4.3 Quelles sont les principales formes de littéraux de chaînes et d'octets ?** Les littéraux str représentent du texte Unicode, bytes des séquences d'octets, les chaînes brutes réduisent l'échappement des barres obliques et les f-strings incorporent des champs évalués.

Engineering Notes / Notes d'ingénierie

EN	FR
• Use underscores in long numeric literals when they improve readability. • A Python 'constant' is usually a naming convention unless an API or type discipline enforces it. • Keep protocol bytes separate from human-readable text.	• Utiliser des traits de soulignement dans les longs nombres lorsqu'ils améliorent la lecture. • Une « constante » Python est généralement une convention de nommage sauf si une API ou une discipline de typage l'impose. • Séparer les octets de protocole du texte destiné à l'utilisateur.

Shared engineering example - code remains English only

```
value = 0b1010_1100
address = 0x4000_2100
mask = 0x00FF_FFFF
packet = b"\xAA\x55\x10\x03"
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 5 - Python 3.15 Language Additions - Overview & Cross-References

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q5.1 Why is version-specific material separated? Language and runtime features can change between development, release-candidate and final releases. Version-sensitive material should be checked against the final official documentation before publication.	**Q5.1 Pourquoi les éléments propres à une version sont-ils présentés séparément ?** Les fonctions du langage et du runtime peuvent évoluer entre les versions de développement, candidates et finales. Les éléments sensibles à la version doivent être vérifiés dans la documentation officielle finale avant publication.
Q5.2 What kinds of additions should be cross-referenced? New syntax, built-ins, import behavior, immutable containers, sentinel facilities, profiling features and ABI/runtime changes belong in the chapters where their semantics are explained in depth.	**Q5.2 Quels types de nouveautés doivent faire l'objet de renvois ?** La nouvelle syntaxe, les built-ins, le comportement des imports, les conteneurs immuables, les sentinelles, le profilage et les changements ABI/runtime doivent être détaillés dans les chapitres concernés.
Q5.3 What is the engineering rule for a new Python feature? Confirm interpreter support, deployment constraints and fallback behavior before using a new feature in production tooling.	**Q5.3 Quelle règle d'ingénierie appliquer à une nouvelle fonctionnalité de Python ?** Avant d'utiliser une nouveauté en production, confirmer la version de l'interpréteur, les contraintes de déploiement et le comportement de repli.

Engineering Notes / Notes d'ingénierie

EN	FR
• Treat 3.15-specific statements as version-sensitive until verified against the final release. • Do not let a convenient new feature silently raise the minimum runtime version. • Record the Python version used by production and validation stations.	• Considérer les éléments propres à 3.15 comme sensibles à la version jusqu'à vérification finale. • Ne pas laisser une nouveauté pratique augmenter silencieusement la version minimale requise. • Documenter la version Python utilisée sur les postes de production et de validation.

Shared engineering example - code remains English only

```
import sys
print(sys.version)
print(sys.version_info)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 6 - Variables, Names & Binding

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q6.1 Does a Python variable contain an object? A name is bound to an object; it is not a fixed typed storage box in the C sense. Assignment changes bindings.	**Q6.1 Une variable Python contient-elle un objet ?** Un nom est lié à un objet; ce n'est pas une case mémoire typée fixe comme en C. L'affectation modifie les liaisons.
Q6.2 What is aliasing? Aliasing occurs when multiple names reference the same object. Mutating that object through one alias can be visible through the others.	**Q6.2 Qu'est-ce que l'aliasing ?** L'aliasing apparaît lorsque plusieurs noms référencent le même objet. Une mutation via un alias peut être visible par les autres.
Q6.3 What is rebinding? Rebinding makes a name refer to another object. It does not modify the object previously referenced by that name.	**Q6.3 Qu'est-ce que la réaffectation d'un nom (rebinding) ?** Le rebinding fait référencer un nom à un autre objet. Il ne modifie pas l'objet précédemment référencé.

Engineering Notes / Notes d'ingénierie

EN	FR
• Assignment does not automatically copy mutable objects. • Use clear names for values with physical units. • Be careful when sharing mutable configuration objects across subsystems.	• L'affectation ne copie pas automatiquement les objets mutables. • Utiliser des noms clairs pour les valeurs associées à des unités physiques. • Être prudent lors du partage d'objets de configuration mutables entre sous-systèmes.

Shared engineering example - code remains English only

```
limits = {"low": 4.75, "high": 5.25}
active_limits = limits
active_limits["high"] = 5.20
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 7 - Objects, Identity, Type & Value

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q7.1 What properties does a Python object have? Every object has an identity, a type and a value. Mutability determines whether its value can change after creation.	**Q7.1 Quelles propriétés possède un objet Python ?** Chaque objet possède une identité, un type et une valeur. La mutabilité détermine si sa valeur peut changer après sa création.
Q7.2 What is the difference between == and is? == compares values according to equality semantics; is compares object identity.	**Q7.2 Quelle est la différence entre == et is ?** == compare les valeurs selon la sémantique d'égalité; is compare l'identité des objets.
Q7.3 Why does mutability matter? Mutable objects can change in place, which affects aliases, function arguments, containers and concurrent code.	**Q7.3 Pourquoi la mutabilité est-elle importante ?** Les objets mutables peuvent changer sur place, ce qui affecte les alias, les arguments de fonctions, les conteneurs et le code concurrent.

Engineering Notes / Notes d'ingénierie

EN	FR
• Use is None for the singleton None. • Do not infer value equality from object identity. • Understand whether an API returns a new object or mutates an existing one.	• Utiliser is None pour le singleton None. • Ne pas déduire l'égalité de valeurs à partir de l'identité des objets. • Savoir si une API renvoie un nouvel objet ou modifie l'objet existant.

Shared engineering example - code remains English only

```
reading = 5.0
print(type(reading))
print(id(reading))
print(reading == 5.0)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 8 - Arithmetic Operators

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q8.1 What are Python's main arithmetic operators? + , - , * , / , // , % and ** provide addition, subtraction, multiplication, true division, floor division, remainder and exponentiation.	**Q8.1 Quels sont les principaux opérateurs arithmétiques de Python ?** + , - , * , / , // , % et ** fournissent addition, soustraction, multiplication, division réelle, division entière par plancher, reste et exponentiation.
Q8.2 How do / and // differ? / performs true division. // returns the floor of the mathematical quotient, which matters especially for negative operands.	**Q8.2 Quelle est la différence entre / et // ?** / effectue une division réelle. // renvoie le plancher du quotient mathématique, ce qui est particulièrement important avec des opérandes négatifs.
Q8.3 Why should engineers be careful with numeric operations? Units, range, precision, overflow expectations in external systems and floating-point representation must be explicit.	**Q8.3 Pourquoi les ingénieurs doivent-ils être prudents avec les opérations numériques ?** Les unités, plages, précision, attentes de débordement dans les systèmes externes et la représentation flottante doivent être explicites.

Engineering Notes / Notes d'ingénierie

EN	FR
• Python integers are not fixed-width machine integers. • Floating-point arithmetic is approximate. • Convert units explicitly and name them in variables.	• Les entiers Python ne sont pas des entiers machine à largeur fixe. • L'arithmétique flottante est approximative. • Convertir explicitement les unités et les indiquer dans les noms de variables.

Shared engineering example - code remains English only

```
power_w = voltage_v * current_a
samples_per_ms = sample_rate_hz / 1000
page = index // records_per_page
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 9 - Comparison Operators

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q9.1 What comparison operators are available? Python supports <, <=, >, >=, == and !=, plus identity and membership operations discussed separately.	**Q9.1 Quels opérateurs de comparaison sont disponibles ?** Python prend en charge <, <=, >, >=, == et !=, ainsi que les opérations d'identité et d'appartenance traitées séparément.
Q9.2 What are chained comparisons? Expressions such as low <= value <= high evaluate as a comparison chain and naturally express bounded ranges.	**Q9.2 Que sont les comparaisons chaînées ?** Les expressions comme low <= value <= high sont évaluées comme une chaîne de comparaisons et expriment naturellement une plage bornée.
Q9.3 Should analog measurements be compared with exact equality? Usually not. Floating representation, noise and tolerance often require a range or tolerance-based comparison.	**Q9.3 Faut-il comparer les mesures analogiques par égalité exacte ?** Généralement non. La représentation flottante, le bruit et les tolérances nécessitent souvent une plage ou une comparaison avec tolérance.

Engineering Notes / Notes d'ingénierie

EN	FR
• Define inclusive/exclusive boundaries explicitly. • Document measurement units and tolerances. • Do not mix value equality with identity.	• Définir explicitement les limites inclusives ou exclusives. • Documenter les unités et tolérances de mesure. • Ne pas confondre égalité de valeur et identité.

Shared engineering example - code remains English only

```
LOW_V = 4.75
HIGH_V = 5.25
passed = LOW_V <= measured_v <= HIGH_V
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 10 - Boolean & Logical Operations

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q10.1 How do and, or and not work? not negates truth value; and and or use short-circuit evaluation and return one of their operands rather than forcing a bool result.	**Q10.1 Comment fonctionnent and, or et not ?** not inverse la valeur de vérité; and et or utilisent l'évaluation en court-circuit et renvoient l'un de leurs opérandes au lieu de forcer un bool.
Q10.2 What is short-circuit evaluation? The second operand is evaluated only when needed to determine the result.	**Q10.2 Qu'est-ce que l'évaluation en court-circuit ?** Le second opérande n'est évalué que s'il est nécessaire pour déterminer le résultat.
Q10.3 Why can truth testing be useful and dangerous? It makes code concise, but values such as 0, empty containers and None can have different engineering meanings and should not be conflated accidentally.	**Q10.3 Pourquoi le test de vérité peut-il être utile et risqué ?** Il rend le code concis, mais 0, les conteneurs vides et None peuvent avoir des significations techniques différentes et ne doivent pas être confondus.

Engineering Notes / Notes d'ingénierie

EN	FR
• Use explicit checks when zero is a valid measurement. • Short-circuiting can safely guard access to dependent data. • Avoid clever Boolean expressions with hidden side effects.	• Utiliser des tests explicites lorsque zéro est une mesure valide. • Le court-circuit peut protéger l'accès à des données dépendantes. • Éviter les expressions booléennes complexes avec effets de bord cachés.

Shared engineering example - code remains English only

```
if reply is not None and len(reply) >= 4:
    process_reply(reply)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 11 - Bitwise Operators & Engineering Bit Manipulation

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q11.1 What are the main bitwise operators? & , | , ^ , ~ , << and >> perform AND, OR, XOR, inversion and shifts on integers.	**Q11.1 Quels sont les principaux opérateurs bit à bit ?** & , | , ^ , ~ , << et >> effectuent AND, OR, XOR, inversion et décalages sur les entiers.
Q11.2 How are masks used? Masks isolate, set, clear or toggle selected bits in registers and protocol fields.	**Q11.2 Comment utilise-t-on les masques binaires ?** Les masques servent à isoler, positionner, effacer ou basculer des bits sélectionnés dans les registres et champs de protocole.
Q11.3 Why does fixed width matter? Python integers have arbitrary precision, while MCU registers and protocol fields have fixed widths. Mask results explicitly when emulating fixed-width behavior.	**Q11.3 Pourquoi la largeur fixe est-elle importante ?** Les entiers Python ont une précision arbitraire, tandis que les registres MCU et champs de protocole ont une largeur fixe. Il faut masquer explicitement les résultats pour reproduire ce comportement.

Engineering Notes / Notes d'ingénierie

EN	FR
• Parenthesize masks for readability. • Document bit numbering and field width. • Validate shifts and external register values before use.	• Parenthéser les masques pour améliorer la lecture. • Documenter la numérotation des bits et la largeur des champs. • Valider les décalages et les valeurs de registres externes avant utilisation.

Shared engineering example - code remains English only

```
STATUS_READY = 1 << 3
ready = bool(status & STATUS_READY)
status |= STATUS_READY
status &= ~STATUS_READY
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 12 - Assignment & Assignment Expressions

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q12.1 What does ordinary assignment do? Assignment evaluates the right-hand side and binds the result to one or more targets.	**Q12.1 Que fait une affectation ordinaire ?** L'affectation évalue le côté droit et lie le résultat à une ou plusieurs cibles.
Q12.2 How does augmented assignment differ? Operators such as += combine an operation with assignment and may mutate an object in place when supported.	**Q12.2 En quoi l'affectation augmentée est-elle différente ?** Les opérateurs comme += combinent une opération et une affectation et peuvent modifier un objet sur place lorsque cela est pris en charge.
Q12.3 What is an assignment expression? The := operator binds a value as part of an expression. It can reduce repeated work but should be used only when it improves clarity.	**Q12.3 Qu'est-ce qu'une expression d'affectation ?** L'opérateur := lie une valeur à l'intérieur d'une expression. Il peut éviter un calcul répété mais ne doit être utilisé que s'il améliore la clarté.

Engineering Notes / Notes d'ingénierie

EN	FR
• Right-hand expressions are evaluated before target binding. • Multiple assignment supports unpacking and safe swapping. • Do not overuse := in safety- or test-critical logic.	• Les expressions du côté droit sont évaluées avant la liaison des cibles. • L'affectation multiple permet le déballage et l'échange sûr de valeurs. • Ne pas abuser de := dans une logique critique de test ou de sécurité.

Shared engineering example - code remains English only

```
a, b = b, a
if (reply := instrument.read()) is not None:
    log_reply(reply)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 13 - Membership & Identity Operations

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q13.1 What do in and not in test? They test membership. For dictionaries, membership tests keys by default.	**Q13.1 Que vérifient in et not in ?** Ils testent l'appartenance. Pour les dictionnaires, le test porte par défaut sur les clés.
Q13.2 When should is and is not be used? Use them when object identity is the intended question, especially with singletons such as None.	**Q13.2 Quand faut-il utiliser is et is not ?** Les utiliser lorsque l'identité de l'objet est réellement la question, notamment avec les singletons comme None.
Q13.3 Why is x in d different from x in d.values()? x in d tests keys; x in d.values() searches the values view. Their semantics and performance differ.	**Q13.3 Pourquoi x in d diffère-t-il de x in d.values() ?** x in d teste les clés; x in d.values() recherche dans la vue des valeurs. Leur sémantique et leurs performances diffèrent.

Engineering Notes / Notes d'ingénierie

EN	FR
• Use sets for fast membership of discrete allowed states. • Do not rely on integer or string interning for value comparisons. • Use a unique sentinel when None is a valid data value.	• Utiliser les ensembles pour tester rapidement des états discrets autorisés. • Ne pas dépendre de l'internement des entiers ou chaînes pour comparer des valeurs. • Utiliser une sentinelle unique lorsque None est une valeur de données valide.

Shared engineering example - code remains English only

```
allowed = {"IDLE", "RUN", "FAULT"}
if state not in allowed:
    raise ValueError("invalid state")
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 14 - Conditional Expressions

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q14.1 What is a conditional expression? value_if_true if condition else value_if_false selects one of two values; only the selected branch expression is evaluated.	**Q14.1 Qu'est-ce qu'une expression conditionnelle ?** value_if_true if condition else value_if_false sélectionne l'une de deux valeurs; seule la branche choisie est évaluée.
Q14.2 Is it the same as an if statement? No. A conditional expression computes a value; an if statement controls execution of statement suites.	**Q14.2 Est-ce la même chose qu'une instruction if ?** Non. Une expression conditionnelle calcule une valeur; une instruction if contrôle l'exécution de blocs d'instructions.
Q14.3 When should it be avoided? Avoid nested or side-effect-heavy conditional expressions when an explicit if/else block communicates intent more clearly.	**Q14.3 Quand faut-il éviter une expression conditionnelle ?** Éviter les expressions conditionnelles imbriquées ou riches en effets de bord lorsqu'un bloc if/else explicite est plus clair.

Engineering Notes / Notes d'ingénierie

EN	FR
• Use conditional expressions for compact value selection. • Keep threshold meaning visible through named constants. • Prefer explicit control flow for logging, cleanup or multiple actions.	• Utiliser les expressions conditionnelles pour une sélection de valeur compacte. • Rendre le sens des seuils visible avec des constantes nommées. • Préférer un contrôle explicite pour la journalisation, le nettoyage ou plusieurs actions.

Shared engineering example - code remains English only

```
result = "PASS" if LOW_V <= measured_v <= HIGH_V else "FAIL"
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 15 - Operator Precedence, Associativity & Evaluation Order

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q15.1 Why is operator precedence important? It determines how an expression is grouped when parentheses are absent.	**Q15.1 Pourquoi la priorité des opérateurs est-elle importante ?** Elle détermine comment une expression est regroupée en l'absence de parenthèses.
Q15.2 What is a notable associativity rule? Exponentiation associates right-to-left: a ** b ** c groups as a ** (b ** c).	**Q15.2 Quelle est une règle remarquable d'associativité ?** L'exponentiation s'associe de droite à gauche : a ** b ** c se regroupe comme a ** (b ** c).
Q15.3 How should dense engineering expressions be written? Use parentheses even when technically unnecessary if they make masks, limits or mixed operations easier to review.	**Q15.3 Comment faut-il écrire les expressions d'ingénierie complexes ?** Utiliser des parenthèses même lorsqu'elles ne sont pas strictement nécessaires si elles rendent les masques, limites ou opérations mixtes plus faciles à vérifier.

Engineering Notes / Notes d'ingénierie

EN	FR
• Re-check precedence when translating expressions from C/C++. • Short-circuit operations intentionally skip operands. • Avoid relying on side effects inside dense expressions.	• Revérifier la priorité lors de la traduction d'expressions depuis C/C++. • Les opérations en court-circuit omettent volontairement certains opérandes. • Éviter de dépendre d'effets de bord dans les expressions denses.

Shared engineering example - code remains English only

```
field = (register >> 8) & 0x0F
valid = enabled and (LOW <= value <= HIGH)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 16 - Simple Statements

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q16.1 What is a simple statement? A simple statement fits within one logical line. Examples include expression, assignment, import, return, raise, pass, del, assert, break, continue, global and nonlocal statements.	**Q16.1 Qu'est-ce qu'une instruction simple ?** Une instruction simple tient dans une ligne logique. Exemples : expression, affectation, import, return, raise, pass, del, assert, break, continue, global et nonlocal.
Q16.2 What does pass do? pass performs no operation; it is a syntactic placeholder where a statement is required.	**Q16.2 Que fait pass ?** pass n'effectue aucune opération; c'est un espace réservé syntaxique lorsqu'une instruction est requise.
Q16.3 Why should assert not replace production validation? Assertions can be removed when Python runs with optimization. Required input and safety checks must use explicit validation and exceptions.	**Q16.3 Pourquoi assert ne doit-il pas remplacer la validation en production ?** Les assertions peuvent être supprimées lorsque Python s'exécute avec optimisation. Les validations requises doivent utiliser des contrôles explicites et des exceptions.

Engineering Notes / Notes d'ingénierie

EN	FR
• Prefer one statement per line in maintainable engineering code. • Use raise for explicit error signaling. • Understand the scope effects of global and nonlocal before using them.	• Préférer une instruction par ligne dans du code d'ingénierie maintenable. • Utiliser raise pour signaler explicitement une erreur. • Comprendre les effets de portée de global et nonlocal avant de les utiliser.

Shared engineering example - code remains English only

```
if value < 0:
   raise ValueError("negative measurement")
return value
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 17 - Compound Statements, Blocks & Suites

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q17.1 What is a compound statement? A compound statement contains or controls suites of other statements, such as if, while, for, try, with, match, function definitions and class definitions.	**Q17.1 Qu'est-ce qu'une instruction composée ?** Une instruction composée contient ou contrôle des blocs d'autres instructions, comme if, while, for, try, with, match et les définitions de fonctions ou classes.
Q17.2 What are a clause and a suite? A clause has a header ending in a colon and a suite controlled by that header. In normal multi-line style, the suite is the indented block that follows.	**Q17.2 Qu'est-ce qu'une clause et une suite d'instructions ?** Une clause possède un en-tête terminé par deux-points et une suite contrôlée par cet en-tête. En style multiligne normal, la suite est le bloc indenté qui suit.
Q17.3 Why is indentation part of control-flow correctness? Indentation determines which operations are conditional, repeated or protected by resource/error-handling logic.	**Q17.3 Pourquoi l'indentation fait-elle partie de la justesse du flux de contrôle ?** L'indentation détermine quelles opérations sont conditionnelles, répétées ou protégées par la gestion des ressources et erreurs.

Engineering Notes / Notes d'ingénierie

EN	FR
• Keep nesting shallow enough to review safely. • Use with for resources that support context management. • Use try/finally when cleanup must be guaranteed and a context manager is not appropriate.	• Garder une profondeur d'imbrication raisonnable pour faciliter la vérification. • Utiliser with pour les ressources compatibles avec les gestionnaires de contexte. • Utiliser try/finally lorsque le nettoyage doit être garanti et qu'un gestionnaire de contexte n'est pas adapté.

Shared engineering example - code remains English only

```
with open("measurements.csv", "w", encoding="utf-8") as f:
    f.write("time,value\n")
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 18 - Conditional Statements

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q18.1 How does if/elif/else execute? Conditions are tested in order. The suite belonging to the first truthy if/elif condition executes; otherwise the optional else suite executes.	**Q18.1 Comment s'exécute if/elif/else ?** Les conditions sont testées dans l'ordre. Le bloc du premier if/elif vrai s'exécute; sinon le bloc else facultatif s'exécute.
Q18.2 When should separate if statements be used instead of elif? Use separate if statements when several independent conditions may all require action. Use elif for mutually exclusive choices.	**Q18.2 Quand utiliser plusieurs instructions if plutôt que elif ?** Utiliser plusieurs if indépendants lorsque plusieurs conditions peuvent toutes nécessiter une action. Utiliser elif pour des choix mutuellement exclusifs.
Q18.3 How should measurement limits be written? Make units and boundary semantics explicit. Chained comparisons such as low <= measured <= high map naturally to an inclusive PASS window.	**Q18.3 Comment faut-il exprimer les limites de mesure ?** Rendre explicites les unités et les limites. Les comparaisons chaînées comme low <= measured <= high correspondent naturellement à une fenêtre PASS inclusive.

Engineering Notes / Notes d'ingénierie

EN	FR
• Order overlapping thresholds carefully. • Make default/failure behavior explicit for interlocks. • Avoid exact floating equality for noisy analog measurements.	• Ordonner soigneusement les seuils qui se chevauchent. • Rendre explicite le comportement par défaut ou de panne pour les interverrouillages. • Éviter l'égalité flottante exacte pour les mesures analogiques bruitées.

Shared engineering example - code remains English only

```
if temperature_c >= 100.0:
    shutdown_output()
    raise RuntimeError("Overtemperature")
elif temperature_c >= 85.0:
    reduce_load()
else:
    maintain_normal_operation()
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 19 - Loops & Iterative Control Flow

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q19.1 What is the difference between for and while? for iterates over items from an iterable; while repeats while its condition remains truthy.	**Q19.1 Quelle est la différence entre for et while ?** for parcourt les éléments d'un itérable; while répète tant que sa condition reste vraie.
Q19.2 What do break and continue do? break exits the nearest loop. continue skips the rest of the current iteration and proceeds to the next iteration.	**Q19.2 Que font break et continue ?** break quitte la boucle la plus proche. continue saute le reste de l'itération courante et passe à la suivante.
Q19.3 What does else on a loop mean? The else suite executes when the loop finishes normally without encountering break. This is useful for search and retry logic.	**Q19.3 Que signifie else associé à une boucle ?** Le bloc else s'exécute lorsque la boucle se termine normalement sans rencontrer break. C'est utile pour les recherches et les tentatives répétées.

Engineering Notes / Notes d'ingénierie

EN	FR
• Use bounded retries and timeouts around hardware communication. • Avoid mutating a collection unpredictably while iterating over it. • Use generators/iterators to stream large measurement sets.	• Utiliser un nombre limité de tentatives et des délais d'attente pour les communications matérielles. • Éviter de modifier de façon imprévisible une collection pendant son parcours. • Utiliser générateurs et itérateurs pour traiter de grands ensembles de mesures en flux.

Shared engineering example - code remains English only

```
for attempt in range(3):
   try:
      reply = read_device(timeout=0.5)
      break
   except TimeoutError:
      log.warning("timeout attempt=%d", attempt + 1)
else:
   raise RuntimeError("device did not respond")
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 20 - Structural Pattern Matching

Engineering focus / Orientation ingénierie: Precise semantics, practical engineering consequences, common pitfalls and review-ready examples. / Sémantique précise, conséquences pratiques, pièges courants et exemples prêts à réviser.

ENGLISH	FRANÇAIS
Q20.1 What is structural pattern matching? match compares a subject against case patterns that can test structure and bind components. It can destructure sequences, mappings and class instances.	**Q20.1 Qu'est-ce que la correspondance de motifs structurels ?** match compare un sujet à des motifs case capables de tester une structure et de lier ses composants. Il peut déstructurer des séquences, mappings et instances de classes.
Q20.2 What is a guard in a case clause? A guard is an additional if condition evaluated after the structural pattern matches. The case is selected only when both succeed.	**Q20.2 Qu'est-ce qu'une garde dans une clause case ?** Une garde est une condition if supplémentaire évaluée après la correspondance structurelle. Le case n'est choisi que si les deux réussissent.
Q20.3 Why is _ special in match patterns? In a pattern, _ is a wildcard and matches without binding a normal capture name. It is commonly used as a final catch-all case.	**Q20.3 Pourquoi _ est-il particulier dans les motifs match ?** Dans un motif, _ est un joker qui correspond sans lier un nom de capture normal. Il est souvent utilisé comme dernier cas général.

Engineering Notes / Notes d'ingénierie

EN	FR
• Patterns are not ordinary expressions. • Use match when data shape matters, such as decoded protocol messages. • Validate captured values with guards when ranges matter.	• Les motifs ne sont pas des expressions ordinaires. • Utiliser match lorsque la forme des données est importante, par exemple pour des messages de protocole décodés. • Valider les valeurs capturées avec des gardes lorsque les plages sont importantes.

Shared engineering example - code remains English only

```
match message:
    case {"cmd": "write", "addr": addr, "value": value} if 0 <= value <= 255:
        write_register(addr, value)
    case {"cmd": "read", "addr": addr}:
        return read_register(addr)
    case _:
        raise ValueError("unsupported message")
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.